

Scenario index

Every scenario, its package, what it teaches, and the one line to remember under exam pressure.

Part 1 — Mocking (EasyMock 5.2.0 + JUnit 5)

#	Package	Technique	The rule
01	ise.mocking.s01_setup_annotations	@ExtendWith(EasyMockExtension.class) + @TestSubject + @Mock	The @TestSubject field must already be instantiated — = new Sut(), not just declared.
02	ise.mocking.s02_setup_manual	createMock() + constructor injection	Works everywhere with nothing to misconfigure. Use it when the SUT has no no-arg constructor.
03	ise.mocking.s03_setup_support	extends EasyMockSupport, replayAll() / verifyAll()	With 3+ mocks, replayAll() removes the silent bug of leaving one mock out of verify(...).
04	ise.mocking.s04_mocktypes	default / strict / nice, side by side	default catches unexpected calls; strict adds order; nice catches nothing you did not record .
05	ise.mocking.s05_stubbing	andReturn, chained, times(n), anyTimes(), andStubReturn	andReturn is never checked by verify() — never use it for a call you must prove happened.
06	ise.mocking.s06_voidmethods	expectLastCall() + times/anyTimes/andThrow/asStub	A void method has no return value, so verify() is the only witness there is.
07	ise.mocking.s07_callcounts	once, times(n), times(min,max), atLeastOnce, anyTimes	anyTimes() includes zero , so on its own it proves nothing.
08	ise.mocking.s08_exceptions	andThrow, propagating vs handling SUT	When the SUT swallows an exception, assert the resulting state — an empty catch block also returns false.
09	ise.mocking.s09_matchers	eq, anyX, isA, isNull/notNull, same, matches, contains, startsWith, lt/gt, and/or/not	One matcher in a call means every argument must be a matcher. Wrap the fixed ones in eq().
10	ise.mocking.s10_capture	newCapture(), capture(), getValue(), CaptureType.ALL	verify() proves the call happened; only a capture proves what was inside the argument.
11	ise.mocking.s11_dynamicreturns	andAnswer() + getCurrentArguments(), andDelegateTo()	Use these when the answer depends on the input; andDelegateTo keeps verify() available.
12	ise.mocking.s12_callorder	createStrictMock	Order is the only thing a default mock cannot see. Reach for strict when the task says "first ... then".
13	ise.mocking.s13_multiplemocks	replay(a, b) / verify(a, b)	A mock missing from verify(...) is never checked, and nothing in the code review shows it.
14	ise.mocking.s14_mustnotbecalled	Proving non-execution	Do not record the call. On a default mock, absent from the record phase == forbidden.
15	ise.mocking.s15_partialmocks	partialMockBuilder().addMockedMethod()	For the one unmockable dependency (clock, random) in an otherwise testable class.

#	Package	Technique	The rule
16	ise.mocking.s16_injectionseams	constructor / setter / field / parameter / factory, plus a SUT with no seam	If the SUT does <code>new Collaborator()</code> inside the method, there is nothing to inject — mock the factory instead.
17	ise.mocking.s17_errorcatalogue	Five classic EasyMock failures, each <code>@Disabled</code> with the exact message	Recognise the message, jump straight to the fix. Delete one <code>@Disabled</code> to see it live.
—	ise.mocking.exam_vehicle	Retake exam, exercise 5 — all four tasks	Every test needs both <code>verify(mock)</code> and a state assertion on the real <code>Driver</code> .
—	ise.mocking.exam_docking	Final exam, exercise 4 ("Mocky")	Same shape, one guard. <code>verify()</code> is what makes the failure test fail on an empty body.

Part 2 — Testing (JUnit Jupiter 5.10.2)

#	Package	Technique	The rule
01	ise.testing.s01_lifecycle	<code>@BeforeAll</code> / <code>@BeforeEach</code> / <code>@AfterEach</code> / <code>@AfterAll</code> , fresh instance per test	<code>@BeforeAll</code> must be <code>static</code> . JUnit builds a new test-class instance per test , so fields are automatically fresh.
02	ise.testing.s02_aaa	Arrange / Act / Assert	One action per test. Two calls in the Act block means a failure will not tell you which one broke.
03	ise.testing.s03_assertions	The complete assertion set	<code>assertEquals(EXPECTED, ACTUAL)</code> — expected first. Doubles always need a delta.
04	ise.testing.s04_exceptions	<code>assertThrows</code> , its return value, <code>assertDoesNotThrow</code>	The lambda holds exactly the call that must throw — and then assert the state did not change.
05	ise.testing.s05_assertall	Grouped assertions	Without <code>assertAll</code> , the first failure hides every later one.
06	ise.testing.s06_parameterized	<code>@ValueSource</code> , <code>@CsvSource</code> , <code>@MethodSource</code> , <code>@EnumSource</code> , <code>@NullAndEmptySource</code> , <code>name=</code>	Put both sides of every boundary in the table; the custom name tells you which row broke.
07	ise.testing.s07_extras	<code>@RepeatedTest</code> , <code>@Timeout</code> , <code>assertTimeout</code> , <code>@Disabled</code> , <code>@DisplayName</code> , <code>@Tag</code>	A timeout assertion still has to assert the result — fast and wrong is still wrong.
08	ise.testing.s08_nested_ordered	<code>@Nested</code> , <code>@TestMethodOrder(OrderAnnotation.class)</code>	Group by state with <code>@Nested</code> . Order tests only when the task describes a workflow.
09	ise.testing.s09_assumptions	<code>assumeTrue</code> / <code>assumeFalse</code>	An assumption skips , an assertion fails . Never use one to hide non-determinism.
10	ise.testing.s10_collections	empty / add / remove / duplicates / order / encapsulation leak	Compare like with like: <code>size()</code> against a number, the list against a list.
11	ise.testing.s11_validation	One test per invalid input class	Also test the valid boundary (zero) — that is the only test an inverted guard breaks.
12	ise.testing.s12_voidstate	The register-a-member pattern	A void method gives you nothing to assert on except the state afterwards .
13	ise.testing.s13_calculations	Normal + both extremes + empty + invalid, with deltas	Never test a formula with 0.0 or 1.0 only — a wrong formula gives the right answer there.
14	ise.testing.s14_query	find / not-found / <code>Optional</code> / filter / sort	A filter test needs the negative assertion, or a filter that returns everything passes.

#	Package	Technique	The rule
15	ise.testing.s15_observers	Hand-written test spy, attach/notify/detach, Consumer<List<T>>	Assert exactly 1 , not "at least 1"; and assert 0 after detaching.
16	ise.testing.s16_contracts	equals / hashCode / toString	Equality tests all pass with a broken hashCode. A HashSet is what exposes it.
17	ise.testing.s17_boundaries	Equivalence classes + boundary values (ATM, overdraft –2000)	For every limit L, test L–1, L, L+1 . > vs >= breaks exactly the L case.
—	ise.testing.exam_library	Retake exam, exercise 4 — all six tests	Assert exact counts . "It did not throw" is worth zero.
—	ise.testing.exam_cart	Final exam, exercise 2 — all five tests	getProductCount() is an int. Do not compare it to a list.

Part 3 — REST (Spring Web + WebFlux)

Package	Covers	The rule
ise.rest.model	Product, Review, sort enums	Jackson needs a no-arg constructor plus getters and setters , or the body arrives empty.
ise.rest.server	ProductResource → ProductService → ProductRepository	Closed layered: the Resource has no repository field . That shortcut costs the architecture marks.
ise.rest.server	POST, GET, GET /{id}, PUT, DELETE, POST /{id}/buy, nested /reviews	DELETE returns 204 No Content . PUT is 400 on an id mismatch, 404 on a missing id.
ise.rest.server	Optional request params	required = false and defaultValue = "false" — with only the first, a missing param is a 500.
ise.rest.server	ProductJpaRepositoryExample (commented)	Derived queries are named, not written: findByQuantityGreaterThan(int).
ise.rest.client	ProductController with WebClient	.subscribe(...), never .block(). Invoke the callback inside the subscribe lambda, after updating the cache.
ise.rest.client	DELETE handling	204 has no body, so bodyToMono(Void.class) never fires onNext — use the completion callback.
ise.rest.exam_pullover	Retake exam, exercise 6	Check existence before quantity : you cannot read the quantity of a stock that is not there.
ise.rest.practice	The same exercise, gutted	Tests are given, server is yours — exactly like Artemis.

Mock exam additions (TEST Exam v2, Jun 2026)

The format changed: no REST exercise, Mocking worth 25 and SOLID worth 25. Full breakdown and the worked text answers are in MOCK_EXAM.md.

Package	Exercise	The rule
ise.mocking.exam_smarthome	Ex. 3 -- Mocking, 25 pts	Both collaborator methods return void , so verify() is the entire test. Two solutions: EasyMock and the hand-written DeviceMock from the exam UML.
ise.solid.exam_enrollment	Ex. 5 -- SOLID, 25 pts	The service depends on EnrollmentRule, never on a concrete rule. No instanceof, ever. Evaluate ALL rules before mutating anything.

Package	Exercise	The rule
ise.blackbox.exam_gamelauncher	Ex. 4 -- Black box, 10 pts	Partition per parameter, one representative per class, then L-1/L/L+1 on every inclusive limit. The value exactly ON the boundary is the one that earns the mark.
ise.acceptance.exam_login	Ex. 1 -- Gherkin, 5 pts	Given = Arrange, When = Act, Then = Assert. Two Given steps and two Then steps minimum, one step per line.
ise.testing.s18_testdoubles	Quiz Q15	STUB feeds values IN; SPY and MOCK check what came OUT; DUMMY is never called; FAKE is a real but simplified implementation.

Errors and omissions in the playbook PDF

I am flagging these because you said you intend to memorise it.

1. `.times(0)` does not exist — and it is recommended twice

MockingPlaybook.java §07:

```
expect(m.charge(7.0)).andReturn(true).times(0); // must NOT be called
```

and §14 "OPTION 2":

```
expect(v.assign(anyObject())).andReturn(true).times(0); // 0 calls allowed
```

Both throw, before the test even reaches `replay()`:

```
java.lang.IllegalArgumentException: maximum must be >= 1
```

EasyMock 5.2.0 validates `1 <= maximum`. `times(0, 0)` fails the same way. **This matters more than any other item here**, because §14 is the "must not be called" case you lost marks on, and one of the three options it offers cannot run.

What actually works — both proved in `ise.mocking.s14_mustnotbecalled`:

- **Do not record the call.** On a default or strict mock, any unrecorded call is an immediate `AssertionError`. This is the answer.
- `andThrow` + `anyTimes()`, if you want the prohibition written out explicitly:

```
expect(mock.forbidden(anyObject()))
    .andThrow(new AssertionError("must not be called"))
    .anyTimes();
```

The `anyTimes()` is essential — without it the expectation becomes mandatory and `verify()` would demand the very call you are forbidding.

2. The PDF is only the two `.java` files

You described it as "(a) a full cheat sheet for the course and (b) two playbooks". It is only (b) — the two playbook listings, reproduced verbatim across 9 pages. There is no separate cheat-sheet section. Nothing is missing from the playbooks themselves; there just is not a third document in there.

3. Things that are correct but read as sharper than they are

- §03 "`createMock ... wrong order → tolerated`" is right, but the playbook does not say that a *strict* mock also fails `verify()` on order violations that happen to be the last call. Scenario 04 demonstrates both directions.
- §18 "**Mental check: if I delete the body of the method under test, does this go red?**" is exactly right, and is the entire premise of Part 4. Note the case the checklist does not warn about: deleting the body often leaves the *state* unchanged in the same way a legitimate failure path would (an empty driver list means both "refused" and "never asked"), so `verify()` is not optional even when you have a state assertion.
- **TestingPlaybook §11** `returnedListDoesNotLeakInternalState` is labelled as a test but is really a demonstration that encapsulation *is* broken — it passes precisely when the leak exists. `ise.testing.s10_collections` keeps that version and adds the `Collections.unmodifiableList` counterpart so the contrast is visible.
- **TestingPlaybook §18** `BlackBoxTests` reuses one `Account` field across tests whose withdrawals would compound if `@BeforeEach` were removed. It is correct as written; `ise.testing.s17_boundaries` keeps the `@BeforeEach` and adds an explicit test that a rejected withdrawal leaves the balance untouched.

Everything else in both playbooks compiles and behaves as documented against JUnit 5.10.2 / EasyMock 5.2.0 / Java 17.